

PROYECTO INTEGRADOR

Generador seguro de Contraseñas

Daniela Loza
1- SOF- A
17-12-2025

¿Por qué un generador seguro de contraseñas?

- Uso constante de datos personales
- Entorno digital
- Necesidad de contraseñas seguras

```

Caracteres a usar para generar una contraseña
MAYUSCULAS = string.ascii_uppercase
MINUSCULAS = string.ascii_lowercase
DIGITOS = string.digits
SIMBOLOS = "@#$%&*+=_"

def generar_contraseña_fija():
    longitud = 12

    # Caracteres obligatorios a usar
    obligatorios = [
        random.choice(MAYUSCULAS),
        random.choice(MINUSCULAS),
        random.choice(DIGITOS),
        random.choice(SIMBOLOS)
    ]

    # Para asegurar que la longitud se cumpla y que todos los elementos estén dentro
    restantes = longitud - len(obligatorios)
    todos = MAYUSCULAS + MINUSCULAS + DIGITOS + SIMBOLOS

    # Para que los caracteres obligatorios se mezclen entre sí
    contraseña_generada = obligatorios[:]
    contraseña_generada += [random.choice(todos) for _ in range(restantes)]
    random.shuffle(contraseña_generada)

    # Para que la lista se convierta en un texto
    return "".join(contraseña_generada)

```


Organización del código

- Código organizado en funciones
- Mejor entendimiento
- Cada parte cumple una tarea



```

# Proceso que el usuario escriba su propia contraseña
def crear_contraseña_usuario():
    # Lo que el usuario debe tomar en cuenta para escribir la contraseña
    print("\nTu contraseña debe tener:")
    print("- Longitud de 12 caracteres (mínimo)")
    print("- Mayúsculas")
    print("- Minúsculas")
    print("- Números")
    print(f"- Símbolos ({SIMBOLOS})")

    # Bucle que se repetirá hasta que la contraseña sea válida
    while True:
        # El usuario escribirá su contraseña aquí
        contraseña = input("\nEscribe tu contraseña: ")

        # Para verificar la contraseña
        faltantes = validar_contraseña(contraseña)

        # Si no le falta nada, la contraseña será válida
        if not faltantes:
            print("Contraseña ingresada válida")
            preguntar_guardar()
            return

        # Si falta algo, se mostrará lo que falta
        print("Su contraseña ingresada no es válida")

```



```

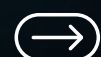
1  import random
2  import string
3
4  # Caracteres a usar para generar una contraseña
5  MAYUSCULAS = string.ascii_uppercase
6  MINUSCULAS = string.ascii_lowercase
7  NUMEROS = string.digits
8  SIMBOLOS = "@#$%&*+= "
9

```

Caracteres

para la generación de contraseñas seguras

- Uso de librerías random y string
- Acceso a letras, números y símbolos
- Generación de contraseñas aleatorias
- Mayor seguridad en las contraseñas



Generación de contraseñas seguras

Generadas
por el
sistema

```

10 # Proceso para que se genere una contraseña automática de 12 caracteres
11 def generar_contraseña_fija():
12     longitud = 12
13
14     # Caracteres obligatorios a usar
15     obligatorios = [
16         random.choice(MAYUSCULAS),
17         random.choice(MINUSCULAS),
18         random.choice(NUMEROS),
19         random.choice(SIMBOLOS)
20     ]
21
22     # Para asegurar que la longitud se cumpla y que todos los elementos estén dentro de la contraseña
23     restantes = longitud - len(obligatorios)
24     todos = MAYUSCULAS + MINUSCULAS + NUMEROS + SIMBOLOS
25
26     # Para que los caracteres obligatorios se mezclen entre sí
27     contraseña_generada = obligatorios[:]
28     contraseña_generada += [random.choice(todos) for _ in range(restantes)]
29     random.shuffle(contraseña_generada)
30
31     # Para que la lista se convierta en un texto
32     return "".join(contraseña_generada)
33
34
35 # Función para validar la contraseña ingresada por el usuario
36 def validar_contraseña(contraseña):
37     # Para verificar el uso de todos los caracteres, sino, mostrar lo que falta
38     faltantes = []
39
40     if len(contraseña) < 12:
41         faltantes.append("longitud mínima de 12 caracteres")
42
43     if not any(c in MAYUSCULAS for c in contraseña):
44         faltantes.append("incluir al menos 1 mayúscula")

```


Generación de contraseñas seguras

Generadas
por el
usuario

```

74
75 # Proceso que el usuario escriba su propia contraseña
76 def crear_contraseña_usuario():
77     # Lo que el usuario debe tomar en cuenta para escribir la contraseña
78     print("\nTu contraseña debe tener:")
79     print("- Longitud de 12 caracteres (mínimo)")
80     print("- Mayúsculas")
81     print("- Minúsculas")
82     print("- Números")
83     print(f"- Símbolos ({SIMBOLOS})")
84
85     # Bucle que se repetirá hasta que la contraseña sea válida
86     while True:
87         # El usuario escribirá su contraseña aquí
88         contraseña = input("\nEscribe tu contraseña: ")
89
90         # Para verificar la contraseña
91         faltantes = validar_contraseña(contraseña)
92
93         # Si no le falta nada, la contraseña será válida
94         if not faltantes:
95             print("Contraseña ingresada válida")
96             preguntar_guardar()
97             return
98
99         # Si falta algo, se mostrará lo que falta
100        print("Su contraseña ingresada no es válida")
101        for f in faltantes:
102            print("-", f)

```

```

105 # Lo que verá el usuario
106 def menu():
107     while True:
108         print("\nGenerador de contraseñas")
109         print("1. Generar una contraseña automática aleatoria")
110         print("2. Crear mi propia contraseña")
111
112         opcion = input("Elige una opción (1 o 2): ").strip()
113
114         if opcion == "1":
115             contraseña = generar_contraseña_fija()
116             print("\nSu contraseña generada es:", contraseña)
117
118         elif opcion == "2":
119             crear_contraseña_usuario()
120
121         # Para evitar otra respuesta a la que el sistema le da a elegir al usuario
122         else:
123             print("Opción no válida. Intente nuevamente.")
124
125
126 menu()
127

```

Menú principal

- Dos opciones
- Ejecución según elección

[Home](#)

[About](#)

[Content](#)

[Others](#)

GRACIAS